

Building this on Vespa — what transfers, and what to re-measure

This is a handover note for the team building library search on Vespa. It is not an argument about the engine: for a real service — hundreds of thousands of holdings, multiple branches, live circulation — Vespa is a reasonable choice and does several things this prototype had to hand-roll.

It is about everything the prototype **measured**, because that is the part a new engine does not give you and the part that is easy to lose in a rebuild. Some of it is counter-intuitive, and one finding runs directly against the thing Vespa makes easiest.

Every number here was produced by code in this repository and can be regenerated. Where a measurement is weak or contested, it says so.

§5a and §5b were added after the rest. §5b matters most if you are about to start: a real Vespa node has since been run over these records, and it broke two of the predictions this note makes about Vespa and falsified one piece of advice badly enough to have sent you the wrong way. Those corrections are marked where the original claim sits, not only in §5b. `vespa/README.md` reproduces the whole thing from a clean clone.

1. What Vespa replaces, and what it does not decide

WHAT THE PROTOTYPE DOES	ON VESPA
BM25 in <code>src/bettersearch/keyword.py</code>	built in
Brute-force cosine over a numpy array	ANN with HNSW (measured in §5b: identical results, and at this corpus size no faster)
Facet counting in <code>api/catalogue.py</code>	grouping
Fixed top-20 cut, then re-rank	phased ranking
<code>sentence-transformers</code> in the API process	an in-cluster embedder
A <code>pgvector</code> table as the production option	not needed

All of that is serving machinery, and Vespa does it better than we did. What Vespa does **not** decide — and what determined quality here — is the list below. Each has a measured answer, and each is a decision your schema and rank profiles will make implicitly whether or not anyone makes it deliberately:

- which encoder, and at what dimensionality
 - what text actually gets embedded
 - where the result list stops
 - whether to fuse keyword and vector lanes, and how
 - what to do about someone typing an exact title or author
 - how you will know whether any of the above helped
-

2. The numbers, as an acceptance bar

41 queries over 4,000 real Open Library records, with 2,378 relevance judgements pooled from six retrieval lanes and judged blind. Each row names the encoder it used, because mixing them is how the gap gets misread:

ARM	NDCG@10
BM25 only	0.443
MiniLM 384d, semantic	0.579
RRF hybrid, BM25 + MiniLM	0.549 — below pure semantic
bge-base 768d, semantic	0.658 — what ships today
bge-base + LLM enrichment of thin records	0.743

A Vespa implementation over the same 4,000 records should reach **0.658** on semantic retrieval. Meaningfully above it, read §5 before celebrating.

The 0.443 BM25 figure is not a port-correctness bar, and an earlier draft of this note was wrong to imply it. It said: below 0.443 on BM25 alone, something in the port has gone wrong. A real Vespa node has since been measured over these records (§5b) and its built-in BM25 scores **0.4120** — below the bar, with nothing wrong. Two reasons, neither of them a defect: Vespa's linguistics stem and ours do not, so it retrieves records our BM25 never surfaced and 25% of its top ten was never judged; and `text` in this corpus is a *derived* concatenation, so the fielded BM25F that Vespa makes possible has nothing extra to weight. Hold the port to **0.658 on dense retrieval**, which reproduces to four decimal places, and treat BM25 as a comparison rather than a gate.

With a reranking phase, the bar is 0.728, which is what the service now ships — a cross-encoder over the enriched text at depth 20, then exact-match promotion. That is the number to hold a Vespa global phase to; §5a has how it is built and the two ways we got it wrong first.

Both figures were re-measured while writing this note and reproduce exactly — 0.6582 and 0.4433, at 100% judged coverage — and again after the pool was widened to six lanes, unchanged. The bar is real, not remembered.

Why a bigger pool left them untouched is worth knowing before you read any nDCG@10 here: 28 of the 41 queries already have ten or more relevant records, so `IDCG@10` is saturated and adding more cannot move the metric. **nDCG@10 is insensitive to pool growth on this eval; recall@10 is not**, since its denominator is every relevant record. That is why `recall@10` is the headline wherever depth or coverage is in play, and it is why the enrichment and encoder figures above survived re-pooling while the reranking figures did not. The one arm of the table not re-measured after re-pooling is `bge-base + enrichment` (0.743) — its index was lost to a container restart and only the MiniLM enrichment arm was rebuilt, which did hold exactly.

The five deliberate known-item controls — exact titles and author names — score **1.000** today. They are a tripwire, not a target: if a change drops them, it has broken exact-name lookup, which is the one thing library users do most.

3. Five traps, each one hit or verified here

A sixth was found later, on the real node, and is in §5b: a rank expression that returns a meaningless score instead of failing.

3.1 Hybrid is the easiest thing to switch on and the thing that lost

Reciprocal rank fusion of BM25 and vector scored **0.549 against pure semantic's 0.579** on this corpus. That result was initially explained away as an artefact of an eval set weighted towards keyword-hostile phrasing, with the prediction that real catalogue traffic — full of exact-name lookups — would be where fusion earned its place. It was re-tested on real records with five deliberate exact-name controls. It did not.

The mechanism is simple: blending two full rankings drags a good list down with a mostly-empty one, because BM25 returns nothing at all for a query that shares no term with any record, and a rank-based fusion still gives those empty positions weight.

This is **not** "do not use hybrid on Vespa". Vespa's phased ranking can express far better combinations than a flat RRF — a vector first phase with a BM25 signal among others in the second phase is a different and more promising shape. It is: **do not ship a hybrid rank profile because the engine offers one.** Measure it against a vector-only profile on the same judgements first.

3.2 Pooling strategy will silently halve your quality

Vespa's Hugging Face embedder defaults to `pooling-strategy: mean`. The `bge` family is trained for **CLS** pooling. Both our server and our browser build use CLS.

Leave the default with a bge model and nothing errors. The service starts, the queries return records, the facets work, and retrieval is quietly worse.

This is worth dwelling on because the same *shape* of failure already cost this project a week: the browser build ran a different encoder from the server, and the only symptom was "the standalone gives different results". There was no error, no warning, and no test that could have caught it, because both sides were individually correct. Any mismatch between how documents and queries are encoded fails this way. Assume you will not notice it; add a check that would.

3.3 Circulation data must stay out of the embedded text

At real scale, availability, copies, due dates and branch change constantly. Bibliographic content does not.

Vespa supports partial updates to attribute fields that leave a tensor field on the same document untouched. Use that: **embed bibliographic content only**, and keep circulation state in attributes updated in place. If availability is part of the embedded text, every return and loan dirties a vector, and at a million holdings that is a re-embedding treadmill that will quietly become your largest compute cost.

The prototype arrives at the same place from a different direction: `content_hash` in `src/bettersearch/chunking.py` is taken over the embedded text only, so re-ingesting a corpus where only availability changed embeds nothing. That property is worth preserving explicitly in the schema rather than rediscovering.

3.4 Filter before the cut, never after

Vespa's filtered ANN does the right thing here, so this is a trap only if you reimplement the obvious order — which is what we did, and it was wrong.

Retrieving the top k by vector and *then* applying a facet does not search the filtered records. It keeps whichever of k already-chosen records happen to match. Measured over 41 queries $\times$ 10 facet values on 4,000 records:

	FILTER AFTER THE CUT	FILTER BEFORE IT
results returned	4.00	19.96
returned fewer than 3	31.2%	0.0%

Each of those facets covers 14–18% of the corpus. **Yours will be far narrower** — "available now, large print, at Ashcombe" is a fraction of a percent of a million holdings — and post-filtering would return an empty page nearly every time. The symptom is not bad ranking; it is a facet panel that appears broken.

Two consequences for the schema and the rank profile:

- Make the filter part of the query, not a step after it. In Vespa that is ordinary — attribute filters combine with `nearestNeighbor` and the engine handles the recall properly, including choosing between pre- and post-filtering strategies. The thing to avoid is doing it in application code on a returned page.
- **Facet counts then have to come from somewhere wider than the page.** If they are counted over a filtered retrieval the sidebar collapses to the value already selected. Vespa grouping computes over the matched set, which is the right answer; just be deliberate about whether a count means "results you would get" or "matches in the corpus", because with a vector query those are very different numbers and only one of them is bounded.

3.5 Make the index carry its encoder's identity

Every index here stores the `model_id` it was built with and checks it on read *and* on write. Change the encoder and the index refuses to answer rather than scoring today's query against yesterday's vectors — which is the failure that returns confident, plausible, wrong results and raises nothing.

At a million records, re-embedding on a model change is a migration measured in hours, run alongside live traffic. The guard is what makes that survivable: whatever the Vespa equivalent is in your deployment — a field on the document, a rank profile that refuses an unexpected tensor dimension, a check in the feed pipeline — decide it before the first model change rather than during it.

4. What changes at real-library scale

The prototype is 4,000 records in one process. Things that did not matter there and will:

- **Cut depth becomes a phase boundary.** We cut the semantic list at a fixed top 20. That number was chosen by sweep — `scripts/tune_cutoff.py` — and fixed top-k beat every similarity threshold tried, because cosine has no absolute floor: on 4,000 records the threshold that looked right on 74 returned a median of 763 results. On Vespa this becomes `rerank-count` and the first phase/second phase split rather than a slice, but the finding stands: **do not cut a vector result list by score.**
- **Facets are counted before filtering.** Ours count over the whole ranked list so the sidebar shows what you *could* narrow to, not what survived the last click. Grouping makes this easy to get either way; it is a product decision worth making deliberately.

- **Multiple collections in one index.** We put 4,000 books and 114 events in one index and measured whether events were both findable and unobtrusive (`scripts/check_events.py`): 12/12 event-shaped queries reach an event, 0/5 known-item lookups show one. One index was enough. With more collection types — articles, archives, databases, e-resources — that balance needs re-checking, and rank fusion across collections is the lever if it tips.
- **Per-branch availability** is the obvious sharding and filtering question and the prototype says nothing useful about it. Ours is invented data.

5. How you will know you are better — and the trap in it

Read this before quoting any score, including ours.

The judgements were produced by pooling: the top 20 from each of six lanes — BM25, MiniLM over thin records, MiniLM over enriched records, bge-base, and the depth-200 `bge-reranker-base` lane plain and exact-match-promoted — unioned and judged blind. A record that no lane retrieved was never shown to a judge, and **an unjudged record counts as irrelevant**.

A Vespa implementation is a *seventh lane*. Every good record it finds that those six missed scores zero. **A better system can therefore score worse.**

Pool it before you quote it. `scripts/build_eval_set.py --lane-file label=rankings.json` takes `{query: [doc_id, ...]}` and judges only what no judge has seen — the last lane cost \$0.06 for 399 records. Carried grades are keyed on query *text*, not index, and the pool only ever grows, so this is safe to repeat.

This is not hypothetical and not small. It happened three times here:

- bge-base first measured *worse* than MiniLM, 0.578 against 0.588, and was nearly rejected. 100% of MiniLM's top 10 had been judged against **71%** of bge's. Re-pooling with bge as a fourth lane added 312 records no other lane had surfaced and moved it **+0.080** — the whole difference between "slightly worse" and "clearly better".
- A BM25 title-weighting change looked harmful for the same reason, and the sweep now reports coverage beside the score so nobody draws that conclusion again.
- Deep reranking measured at **-0.062** at rerank depth 200 on 59.5% coverage. The obvious fix — discard unjudged candidates, pin coverage at 100% — read **+0.087** and was believed. The judged truth is **+0.010**. Both estimators were wrong, in opposite directions, and the second was wrong by more. §5a has the detail; the lesson is that *correcting* for coverage is not the same as *having* the judgements.

And it is reproducible in two minutes, which is the clearest way to see it. Adding one more collection to the index — 114 community events alongside the 4,000 books — produces this:

INDEX	NDCG@10, SEMANTIC	JUDGED COVERAGE
4,000 books (the pooled corpus)	0.6582	100%
the same books + 114 events	0.6511	97.6%

The score fell by 0.007. But coverage fell too, because event records were never in the pool and every one that surfaces is scored as irrelevant by definition. Some of that 0.007 is the events genuinely

displacing books, and some is an artefact of the judgements — **and this eval cannot tell you which**. On the keyword lane the same move takes coverage from 100% to 89.3%.

That is a 114-record change. A new engine over a real catalogue will move coverage far more than that.

So: **scripts/score_rankings.py** reports judged coverage next to every score.

```
python scripts/score_rankings.py vespa-semantic.json
python scripts/score_rankings.py ours-semantic.json vespa-semantic.json # compared
```

It takes the simplest thing any engine can emit — a JSON object mapping each query to its ranked `doc_ids` — and reports `nDCG@10`, `Recall@5`, `MRR@10`, the known-item controls, events intrusion, the weakest queries, and coverage. It uses the same metric functions as everything else in this repository, so a Vespa number and ours are the same arithmetic by construction rather than by agreement.

Reading the coverage line:

COVERAGE OF YOUR TOP 10	WHAT IT MEANS
≥ 90%	the comparison is fair; believe the score
75–90%	treat with caution; the score is probably an undercount
< 75%	not comparable — you are measuring the pool, not the engine

If coverage comes back low, the honest move is to re-pool and re-judge rather than accept the number. `scripts/build_eval_set.py` does this and takes a `--lane` per system, so Vespa can be added as a lane and the whole set re-judged for a few pounds of model time.

What it cannot tell you: anything about latency, cost, freshness, or behaviour at a scale 4,000 records cannot reach. It is a relevance bar and only that.

5a. Where the quality actually is — measured since this note was written

You asked whether Vespa could make meaning search better. It can, and the lever is the reranking phase. Here is the measurement, so the rank profile can be designed against numbers rather than hope.

The ceiling first. Reordering only the candidates we already retrieve, at depth 20 where judged coverage is 100%:

	NDCG@10
retrieval alone	0.6582
what we ship now — reranked at depth 20	0.7280
perfect reordering of the same 20 candidates	0.9124

The middle row is new since this note was first written. Reranking used to live only in a measurement script; it is now in `api/catalogue.py`, on by default, and the figure above is `run_search` scored against the judged queries rather than the script. So the acceptance bar in §2 has moved: a Vespa implementation should reach **0.658 on retrieval alone** and **0.728 with its global phase doing what ours does**.

The retrieval is fine. 161 judged-relevant records across 41 queries are retrieved and sitting at ranks 11–20. At depth 20 reranking never changes the candidate set, so §5's trap does not apply. **Deeper it does**, and §5's warning is not decoration — it cost us two wrong answers before we paid to re-pool. See *Depth* below.

What reached the ceiling, and what did not:

ARM	NDCG@10	CONTROLS	MS/QUERY
baseline	0.6582	1.000	0
BM25 as a weighted second phase	0.6782	1.000	0
cross-encoder over the records as they are	0.6573–0.6632	1.000	186–890
cross-encoder over enriched records	0.7149–0.7257	0.877–0.926	356–2,809
the same, plus exact-match promotion	0.7308	1.000	2,809
late interaction, our pylate MaxSim, enriched	0.7407	1.000 at depth 20	764
late interaction, Vespa native, unenriched	0.7131	1.000	45
oracle	0.9124	1.000	—

The last two rows were added after this note was first written, and the second of them is the one to look at: it needed no enrichment, no exact-match repair and no GPU. §5b has it. The 0.7407 row is ours and carries a caveat the Vespa row does not — it rests on a MaxSim written by hand against `pylate`, which is exactly why it was handed over rather than shipped.

Three things follow, and all three are rank-profile decisions:

1. **A cross-encoder is worth nothing on a bare catalogue record.** +0.005 and –0.001. The record is a 24-word metadata stub and a passage-reading model has nothing to read. The same model separates cleanly on a written sentence, so this is the data, not the model.

An earlier draft of this note generalised that to **"if you put a reranker in the global phase without giving it prose, you will pay the latency and get nothing."** That is false, and it is false in the direction that would stop you trying the thing that worked best. A *late-interaction* reranker scores by matching tokens, not by reading a passage, and on these same 24-word stubs Vespa's native `colbert-embedder` with MaxSim in `second-phase` is worth **+0.0549** — 0.6582 to 0.7131, both arms at 100% judged coverage, 45 ms. Narrow the warning to cross-encoders; see §5b.

2. **Enrichment unlocks it — for a cross-encoder.** At 95 words median the same reranker is worth +0.073. For a cross-encoder, reranking and enrichment are the same bet that these records are too thin, and they compound. For late interaction they are closer to **alternatives**: Vespa's native embedder on unenriched records lands within 0.002 of what our MiniLM reaches only after all 4,000 records have been enriched (+0.0549 against +0.0567). That makes enrichment a budget question rather than a prerequisite.

3. **A cross-encoder breaks exact-name lookup, and that must be caught.** Both enriched arms regressed the known-item controls — Sue Monk Kidd –0.369 — because a model reasoning over prose optimises aboutness at the cost of identity. A rule that lifts named records regardless of score restores them to 1.000 *and* raises the mean.

Late interaction does not need that repair at shallow depth: controls hold at **1.000 natively** under Vespa's ColBERT at `rerank-count 20`, and under our own at depth 20 and 50. They do fall to **0.926 at depth 100**, so it degrades the same way, just later. Either way the advice is unchanged and is the part to keep: **make the controls a gate in CI**, not a thing somebody notices later.

The cost is the catch in Python, and much less of one in Vespa. `bge-reranker-base` is 2.8 s/query on CPU for 20 candidates. In our service that needs ONNX, a GPU, or the MiniLM cross-encoder — which gets most of the gain for **13% of the time**: 96% of it on `nDCG@10` (+0.070 against +0.073) and 89% on `recall@10` (+0.058 against +0.066). If you ship one reranker *in a Python service*, ship that one; it is the one we ship.

An earlier draft said "Vespa pays that on every search". Measured, it does not: the ColBERT `second-phase` arm is a **45 ms** round trip including retrieval, and 70 ms at `rerank-count 200` (§5b). So the recommendation inverts for a Vespa deployment — in-cluster late interaction beats MiniLM-after-enrichment on quality *and* costs a fraction of it, because the model runs where the documents already are and the token tensors are an attribute rather than a per-query model call.

Two numbers, because they measure different things. The arms table above reports **356 ms** for MiniLM — that is `scripts/tune_reranking.py`'s per-arm cost. Measured through a running server, warm, over twelve queries with reranking on and off, the incremental cost is **+371 ms median** (184 ms against 555 ms). Neither is Render's hardware and neither is yours; measure it on the node that will run it.

Load the model at startup, not on first use. Ours lazily loaded on the first search and that search took **85 seconds**, on a box where the weights were already cached — a Hub round-trip plus construction, so pre-fetching during the build does not save you. Whatever Vespa's equivalent is, pay it before the health check passes.

Two implementation details that cost us a cycle each, so you do not have to. Reranking must run **before** exact-match promotion — reversed, it pushes the named records straight back down and the controls land at 0.877. And a cross-encoder's output must not reach the response as a relevance score: it is an uncalibrated logit, not a cosine, and anything downstream reading that field will be quietly wrong. We keep the retrieval score and label the card instead.

Depth: set `rerank-count` to 20. This is the headline correction to an earlier draft of this note, which told you quality does not fall off with depth. It does, monotonically:

RERANK DEPTH	20	50	100	200
<code>recall@10</code> vs baseline	+0.066	+0.047	+0.035	+0.010
judged coverage	100%	92.7%	98.8%	100%
ms/query (CPU)	2,809	5,548	10,093	19,937

Seven times the latency for a seventh of the gain. Rerank the twenty you display.

Vespa reaches the same conclusion from a weaker measurement, and the difference is instructive. Its native ColBERT sweep over the same depths gave 0.7131 / 0.6557 / 0.6449 / 0.6473 — the same shape — but with judged coverage falling *monotonically*, 100% → 89.5% → 87.3% → 85.6%, because reranking deeper promotes records no judge ever saw. The coverage row above is **not** monotonic (100 / 92.7 / 98.8 / 100), and that is the tell: the `+0.010` at depth 200 on 100% coverage cell is one the pool-thinning explanation cannot reach, so the decay measured here is real and not an artefact. The Vespa

sweep's deeper rows are the unreliable ones. Set `rerank-count` to 20 — but if you re-measure it yourself, pool the arm first or only the shallowest row will mean anything.

How that draft got it wrong is the most useful thing in this section, because it is a trap any team measuring a Vespa rank profile will meet. The first sweep, pooled from four lanes, read `+0.079 / +0.029 / -0.007 / -0.062` with coverage falling to 59.5% — §5's bias, clearly. So it was re-run with unjudged candidates *discarded*, pinning coverage at 100%, which read `+0.079 / +0.072 / +0.091 / +0.087` and was written up here as the true shape.

That correction was the bigger error. Discarding unjudged candidates does not remove the bias, it inverts it: what gets discarded is exactly the junk the reranker promoted and nobody graded, so the arm is scored only where it was already known to be doing well. It overstated the gain roughly ninefold at depth 200 — `+0.087` against a judged `+0.010`.

We tested that rather than asserting it. If `--judged-only` is biased by precisely the coverage it throws away, it must agree with the judged truth wherever coverage is already 100%, and over-read everywhere else:

RERANK DEPTH	BGE + PROMOTION — THE LANE WE POOLED			MINILM + PROMOTION — NOT POOLED		
	judged truth	--judged-only	gap	judged truth	--judged-only	gap
20	+0.0657	+0.0657	0.0000	+0.0584	+0.0584	0.0000
50	+0.0465	+0.0596	+0.0131	+0.0308	+0.0680	+0.0372
100	+0.0349	+0.0384	+0.0035	+0.0292	+0.0458	+0.0166
200	+0.0097	+0.0097	0.0000	+0.0249	+0.0419	+0.0170

It agrees to four decimals at exactly the two places it should, and nowhere else. Depth 200 is the one to look at: the bge arm converges **because that is the lane we pooled and judged**, while MiniLM — same corpus, same depth, never pooled — still over-reads by +0.017.

If you take one thing from §5 and §5a, take this: the fix for a coverage problem is judgements on the arm you are measuring, not a filter that makes the coverage number look better.

Neither estimator is safe. Only judgements are. We pooled the depth-200 reranked lane and graded the 399 records it surfaced that no judge had seen, for \$0.06; 81 were relevant; coverage went to 100% and the number stopped moving. `data/rankings/` holds that lane so you can reproduce the pool, and `scripts/bound_repool_effect.py` bounds the gap before you spend — read its ordering, not its margins, which were 3× out. `vespa/rankings/` holds thirteen more, from the real node, **none of them pooled** — see §5b.

Still unexplained: the best arm captures **32%** of the available headroom (0.7407 against an 0.9124 oracle over an 0.6582 baseline; the 0.7308 cross-encoder arm captures 29%, and Vespa's native late interaction 22% while paying for no enrichment at all). The remaining ~0.17 is real, measured, and nobody here knows what reaches it. That is the most interesting open question in this handover, and the first one worth spending a week on.

Reproduce with `python scripts/tune_reranking.py`.

5b. What a real Vespa node measured — and which of my predictions it broke

Everything above §5b was reasoned about Vespa from the outside: read off its documentation and argued against our Python measurements. Eight such claims were worked out for this project and **none of them had been run**. Most were never even written into this note; they lived in conversation. This section records them with numbers for the first time.

One node from `ghcr.io/vespa-engine/vespa`, the same 4,000 records, the same 41 judged queries, scored through `scripts/score_rankings.py` — the same harness behind every figure above. Reproduce it from `vespa/README.md`; the application package is `vespa/app/`, the lane files are `vespa/rankings/`.

Read the coverage column before the nDCG column. These thirteen lanes were never pooled — the judge needs an API key and none was available — so anything a Vespa profile surfaced that our six pooled lanes missed counts as irrelevant. Every figure here is a **floor, not an estimate**, except where coverage is 100%. The comparisons to trust are Vespa against Vespa, where both sides are equally unjudged and the difference is close to unbiased. That is the rule §5's coverage table is missing, and it is what makes the ColBERT A/B below sound.

The port is faithful, which is what licenses the rest

LANE	NDCG@10	COVERAGE	CONTROLS
dense, fed vectors	0.6582	100.0%	1.000

Exactly the §2 bar, to four decimal places. The vectors were not recomputed — they were lifted out of the index behind the published figure, so the only variable is Vespa's retrieval and ranking, and `vespa/scripts/make_feed.py` refuses to write a feed until it has checked all 4,000 records were embedded from exactly the text our encoder saw. Vespa's exact `nearestNeighbor` over an attribute with no `index` block reproduces our brute-force numpy scan. HNSW was deliberately kept out of the gate so that parity meant one thing; it is measured separately below.

And Vespa's own encoder agrees with ours. The row above is fed vectors, so that the only variable was Vespa's retrieval. Moving the encoder inside Vespa too — `hugging-face-embedder` over the ONNX export of `bge-base-en-v1.5`, embedding both the records and the query — gives 0.6582 / 0.2819 / 0.7820 again, and not merely in aggregate: **the same documents in the same order in all 41 top tens**, with identical top-100 ordering on 37 of 41 and 100% set overlap at both depths. The four that differ do so only past position ten. So there is no ONNX-against-torch caveat to carry.

Two of its settings are not defaults and both are load-bearing: `pooling-strategy` must be `cls`, because `bge` pools the CLS token and Vespa defaults to mean; and there must be **no** `<prepend>`, because our encoder adds no instruction prefix to a query and a prefix on one side only would measure the prefix. Get either wrong and you will be measuring the configuration rather than the port. The cost is one forward pass per query: 72 ms round trip against 23 ms with the vector supplied.

The approximation: measured, and the only unbiased number here

Every dense figure in this note used exact search. A library at 500,000 holdings will not.

`vespa/scripts/compare_approximation.py` asks the same index the same question twice, exactly and approximately, and reports how much of the exact answer the approximation found — **no judgements involved**, so unlike every other figure in §5b this one carries no coverage caveat at all.

HNSW.EXPLOREADDITIONALHITS	RECALL@10	RECALL@100	IDENTICAL TOP TEN	MEDIAN LATENCY
exact (reference)	—	—	—	17 ms
0	0.9951	0.9654	40/41	18 ms
100	0.9976	0.9944	40/41	18 ms
500	1.0000	1.0000	41/41	17 ms

At 4,000 records the approximation buys nothing — 17 ms either way. HNSW earns its place by making a scan cheaper, and a scan over $4,000 \times 768$ floats is not expensive. Do not quote a latency win from a corpus this size.

Growing the corpus in stages, with `exploreAdditionalHits` at 0 so the trend is not hidden behind a perfect score: `recall@100` falls monotonically — 0.9793 at 1,000 documents, 0.9702 at 2,000, **0.9639 at 4,000** — while exact latency rises 11 → 14 ms and the approximate stays flat. That is the right shape, over far too small a range to be a scaling law. `recall@10` does not move monotonically and should be read as noise at 41 queries.

The operational warning worth more than the numbers. Vespa falls back to an exact scan when it judges the approximation unhelpful, and says nothing about it. Near-perfect recall therefore has two explanations and the result alone cannot separate them. Check that recall *moves* when you change `exploreAdditionalHits`; a flat sweep means you are measuring a fallback, not a graph. Two bugs on our side produced exactly that flat sweep before it was right: the parameter is an annotation on the operator and is silently ignored as a query property, and our own body builder consumed it from the dict it was handed, so one dict reused across a loop set it on the first query only.

Run it against your own corpus at your own scale. It needs no relevance judgements, so it works on day one.

Confirmed

- **RRF loses, in Vespa's own implementation.** §3.1 predicted this and it holds: `reciprocal_rank_fusion()` in a `global-phase` scores **0.5620** against dense-only's 0.6582, both at coverage §5 calls comparable. Fusing costs 0.096.
- **Partial attribute updates work as §3.3 says.** An `assign` to `available` takes **20 ms** and the filter count moves on the very next query, with the 768-float vector and the ColBERT token tensors untouched. §3.3 cited the mechanism from documentation; it can now cite a run (`vespa/scripts/demo_partial_update.py`).
- **Late interaction**, which is the result worth acting on and is folded into §5a above rather than repeated here.

Overstated: per-node BM25 significance

The prediction was that Vespa builds significance per content node, so a multi-node port would miss the §2 BM25 bar for reasons unrelated to the port. **At this corpus size it will not.**

`vespa/scripts/probe_significance.py` reads the significance Vespa is really using per query term out of `term(i).significance` in `match-features`, recovers each term's document frequency, resamples it from `Binomial(df, 0.5)` to simulate a node holding half the corpus, and re-issues every query through Vespa's own per-term `{significance: x}` annotation.

```
significance ~ 0.7454 + -0.02617 * log(df)      worst residual 0.0254
half-corpus node, 5 random splits:  -0.0009 -0.0051 -0.0012 -0.0011 +0.0002
mean shift -0.0016 nDCG, mean top-10 overlap with global significance 98.0%
```

Significance spans only 0.574 to 0.687 across a **124-fold** range of document frequency — Vespa compresses it hard — so halving the documents a node sees moves a term by about 0.018, which is less than the scatter in the relationship itself. Stated with its limit: this shows the effect is **not large** and cannot show it is zero, because the document-frequency recovery rests on a fit whose residuals are the size of the effect.

The method matters more than the number. `term(i).significance` in `match-features` is how your team can check this on a real multi-node cluster in an afternoon, without guessing how Vespa stemmed anything. The CLI's `significance` subcommand would be the cleaner route; the version matching this server did not have it.

Oversold: binary quantisation

LANE	NDCG@10	COVERAGE	TOP-10 SAME AS FLOAT	ROUND TRIP
float, exact	0.6582	100.0%	—	23 ms
packed, Hamming only	0.5847	83.2%	59.0%	10 ms
packed, then rescored on the floats	0.6580	99.8%	94.4%	10 ms

768 floats at 4 bytes become 96 bytes with `binarize | pack_bits`: a 32-fold reduction. Retrieve on the packed vectors and rescore the survivors with the float ones and essentially all the quality comes back — an identical top ten on 32 of 41 queries, at under half the latency.

But the two benefits are not simultaneous, which is the part the prediction missed. The rescore needs the float vectors, so they must still be stored: that profile buys query cost, not memory. Drop the floats and take the real 32-fold saving and you are at 0.5847 — down 0.07, with 59% of the top ten changed. Free of *latency* here, not of memory. (Matryoshka truncation was not tested: `bge-base-en-v1.5` has no Matryoshka objective, so truncating it would measure the truncation. `README.md`'s 6× storage claim combines it with int8 and quotes no quality cost; treat that as unmeasured.)

Wrong: `sameElement` for subject headings

It needs an array of structs. `subjects` is a flat `array<string>`, so `sameElement` does not apply to this schema at all. The version that is true here is **matched-elements-only**, declared on the field in `vespa/app/schemas/record.sd`: Vespa will return *which* subject headings matched. That is worth knowing because `api/catalogue.py` spends a query-time embedding pass guessing exactly that, to populate `closest_headings` on a card.

A sixth trap, and this one fails silently

The obvious way to rescore binary retrieval on the float vectors is `closeness(field, embedding)` in a second-phase. It scores **0.0837**.

`closeness` only has a distance for the field `nearestNeighbor` actually ran against — here `embedding_binary`. Asked about any other field it returns a meaningless value rather than failing. Write the dot product out instead: `sum(query(q) * attribute(embedding))`.

What makes it worth a trap of its own is how it hides. The giveaway was not the metric but an overlap check: **63.9% of its candidates matched the float lane while its top ten matched 4.1%** — retrieval was right, ordering was noise. Reading nDCG alone would have recorded binary quantisation as catastrophic and moved on. When you measure a rank profile, check set overlap against a known-good profile as well as the metric; they fail in distinguishable ways.

Not measurable here: grouping

§3.5 and §4 say grouping computes facet counts over the matched set, "which is the right answer". Probably true and still unverified: **none of the 41 eval queries carries a facet filter**, so there is no number to move. It would remove the fixed candidate pool `api/catalogue.py` admits is "judgement, not measurement", but that is a demonstration, not a measurement, and this note should not claim otherwise.

A negative result nobody predicted

`src/bettersearch/keyword.py` records that our BM25 cannot weight the author or the subject headings at all, because neither is a field — both exist only as lines of prose inside one `text` blob. Vespa indexes them separately, so this was the first chance to ask whether that matters. **It does not help here.**

LANE	NDCG@10	COVERAGE	CONTROLS
BM25, flat title + text	0.4120	75.4%	0.886
BM25F, best of 27 weightings, fitted	0.3952	75.1%	0.726
BM25F, equal weights	0.3734	72.2%	0.600

All 27 combinations scored below flat, and the best is fitted on the same 41 queries it is scored on, so the out-of-sample number is worse still. The reason is our data, not Vespa: `text` is a derived concatenation that already contains the author line and the subject headings, so weighting them separately double-counts terms `bm25(text)` has scored already. **Fielding would pay on a corpus where the fields carry distinct content.** Check that before budgeting for a fielded schema.

The sweep did reproduce our title-weight finding independently, and turned up something useful: at title weight 2 or above the known-item controls reach **1.000 natively**. Vespa gets there with a rank-profile weight where we needed a separate exact-match promotion stage — though it costs nDCG, the same trade `scripts/tune_title_weight.py` measured.

6. Things worth not rebuilding

Two small pieces earned their place for reasons that are not about ranking, and are cheap to carry over:

- **Exact-match promotion.** A model-free pass over title and author that catches what neither lane can see — word order. It is worth almost nothing as ranking (nDCG 0.658 either way) and ships anyway, because it makes a *guarantee*: a record whose exact title someone typed is on page one because a rule says so, not because the encoder agreed. It also gives the card something true to say — "by this author" is legible in a way a cosine score is not.
- **Saying why a record is there.** Keyword results name the query words the record does *not* contain; meaning-based results name the record's own subject headings closest to the query. Both are cheap, and they are most of what makes a meaning-based result list feel trustworthy rather than arbitrary.

7. Open questions for your team

1. What is the actual corpus — holdings only, or articles, archives and e-resources too? The one-index finding was measured on two collections, not five.
2. Is enrichment of thin records on the table? It is worth +0.085 nDCG on top of the best encoder, and it is the second thing to try, not the first — a bigger encoder buys most of what it buys, for free. And there is now a third option that did not exist when this was written: native late interaction gets most of the enrichment win with no enrichment bill at all (§5b). If the answer to this question is "no, we cannot fund enrichment", that is the route.
3. What is the re-embedding plan when the encoder changes, and who notices if documents and queries fall out of step?
4. Will you re-pool the judgements with Vespa as a lane before comparing? If not, §5 says what the comparison is worth. This is now the single most valuable thing you could spend \$0.06 on: thirteen Vespa lanes are committed in `vespa/rankings/` and **none of them is pooled**, because no API key was available. Every figure in §5b is a floor until they are.
5. Multilingual? Every measurement here is English-only and the encoder choice would change.